

Thème:
MISE EN PLACE D'UN RUNTIME POUR L'EXECUTION DE
CALCULS SUR MACHINES VOLONTAIRES DANS UN
ENVIRONNEMENT A RESSOURCES LIMITÉES

Rapport de travail hebdomadaire

Par NGNAPA NGOULE Ashley

Department d'informatique, Université de Yaoundé 1

21 Novembre 2025

Sommaire

- ① Les tâches de la semaine
- ② Exploring the support of HPC in a container runtime environment
 - subsection I-1
- ③ An Ultra-lightweight Container that Maximizes memory Sharing and Minimizes the Runtime Environment
 - subsection II-1
- ④ Les tâches à faire

Les tâches de la semaine

Les tâches a faire

Le travail de cette semaine s'est articulé au tours des points suivants

- La lecture et le résumé de l'article "Exploring the support of HPC in a container runtime environment"
- La lecture et le résumé de l'article "An Ultra-lightweight Container that Maximizes memory Sharing and Minimizes the Runtime Environment"
- L'implementation de fonctionnalités sur l'application volontaire
 - Ajout de la Gestion des clients
 - ajout de la Validation des clients

Les tâches a faire

Le travail de cette semaine s'est articulé au tours des points suivants

- La lecture et le résumé de l'article "Exploring the support of HPC in a container runtime environment"
- La lecture et le résumé de l'article "An Ultra-lightweight Container that Maximizes memory Sharing and Minimizes the Runtime Environment"
- L'implementation de fonctionnalités sur l'application volontaire
 - Ajout de la Gestion des clients
 - ajout de la Validation des clients

Les tâches a faire

Le travail de cette semaine s'est articulé au tours des points suivants

- La lecture et le résumé de l'article "Exploring the support of HPC in a container runtime environment"
- La lecture et le résumé de l'article "An Ultra-lightweight Container that Maximizes memory Sharing and Minimizes the Runtime Environment"
- L'implementation de fonctionnalités sur l'application volontaire
 - Ajout de la Gestion des clients
 - ajout de la Validation des clients

Exploring the support of HPC in a container runtime environment

Objectif et source de l'article

Objectif de l'article

les principaux objectifs de cet article étaient d'examiner la faisabilité et les performances des technologies de conteneurisation, notamment CoreOS Rkt, pour les applications de calcul haute performance (HPC).

Cet article de Martin, J.P., Kandasamy, A., Chandrasekaran, K. a été publié en 2018 dans le journal Human-centric Computing and Information Sciences

Contexte et enjeux

L'article s'inscrit dans l'évolution du cloud computing et de la virtualisation pour les applications HPC.

Puisque la technologie de conteneurisation la plus utilisée depuis 2013 est Docker malgré des limitation au niveau de la sécurité et de l'exécution de calculs de haute performance, les auteurs se sont donné pour mission de tester les performances de Rkt, un autre outil de conteneurisation, puis comparer à celles de Docker et de LXC.

Le principal enjeu ici est l'étude des performances dans un environnement HPC.

Question de recherche

La question de recherche fondamentale découlant de cet article peut être résumée en deux grands points

- la faisabilité du conteneur CoreOS Rkt pour l'exécution d'applications d'informatique haute performance (HPC)
- dans quelle mesure cette technologie récente, plus légère et plus sécurisée que ses prédécesseurs Docker et LXC

Trois étapes pour répondre à cette question :

- Contexte et travaux connexes
- Outils de configuration expérimentale et d'évaluation comparative (benchmarking)
- Résultats et discussion

Etat de l'art ou contexte et travaux connexes

les auteurs ont réalisé une serie de lectures pour arriver a leurs fins

- la comparaison VM vs conteneurs
- les travaux de Travaux de Chung et al. toujours sur la comparaison VM vs conteneurs, travail qui a légitimé l'accent mis par les auteurs sur l'étude des conteneurs (plutôt que sur une réévaluation des machines virtuelles) pour répondre aux défis du HPC
- Pertinence des Conteneurs dans l'Environnement HPC, article de Xavier et al, qui concluait que LXC est la meilleure technologie de conteneurisation pour l'exécution des calculs HPC.
Ceci leur a permis d'inclure LXC parmi les technologies à comparer
- Travaux de Varma et al. qui les ont permis de comprendre les goulots d'étranglement spécifiques (réseau I/O) rencontrés par Docker
- Travaux de Peinl et al. Au travers desquels ils ont compris que docke présentait sécurité et interopérabilité comme principale faiblesse, et qui par ailleurs étaient la raison directe de l'introduction de Rkt par CoreOS.

Etat de l'art ou contexte et travaux connexes

les auteurs ont réalisé une serie de lectures pour arriver a leurs fins

- la comparaison VM vs conteneurs
- les travaux de Travaux de Chung et al. toujours sur la comparaison VM vs conteneurs, travail qui a légitimé l'accent mis par les auteurs sur l'étude des conteneurs (plutôt que sur une réévaluation des machines virtuelles) pour répondre aux défis du HPC
- Pertinence des Conteneurs dans l'Environnement HPC, article de Xavier et al, qui concluait que LXC est la meilleure technologie de conteneurisation pour l'exécution des calculs HPC.
Ceci leur a permis d'inclure LXC parmi les technologies à comparer
- Travaux de Varma et al. qui les ont permis de comprendre les goulots d'étranglement spécifiques (réseau I/O) rencontrés par Docker
- Travaux de Peinl et al. Au travers desquels ils ont compris que docke présentait sécurité et interopérabilité comme principale faiblesse, et qui par ailleurs étaient la raison directe de l'introduction de Rkt par CoreOS.

Etat de l'art ou contexte et travaux connexes

les auteurs ont réalisé une serie de lectures pour arriver a leurs fins

- la comparaison VM vs conteneurs
- les travaux de Travaux de Chung et al. toujours sur la comparaison VM vs conteneurs, travail qui a légitimé l'accent mis par les auteurs sur l'étude des conteneurs (plutôt que sur une réévaluation des machines virtuelles) pour répondre aux défis du HPC
- Pertinence des Conteneurs dans l'Environnement HPC, article de Xavier et al, qui concluait que LXC est la meilleure technologie de conteneurisation pour l'exécution des calculs HPC.
Ceci leur a permis d'inclure LXC parmi les technologies à comparer
- Travaux de Varma et al. qui les ont permis de comprendre les goulots d'étranglement spécifiques (réseau I/O) rencontrés par Docker
- Travaux de Peinl et al. Au travers desquels ils ont compris que docke présentait sécurité et interopérabilité comme principale faiblesse, et qui par ailleurs étaient la raison directe de l'introduction de Rkt par CoreOS.

Etat de l'art ou contexte et travaux connexes

les auteurs ont réalisé une serie de lectures pour arriver a leurs fins

- la comparaison VM vs conteneurs
- les travaux de Travaux de Chung et al. toujours sur la comparaison VM vs conteneurs, travail qui a légitimé l'accent mis par les auteurs sur l'étude des conteneurs (plutôt que sur une réévaluation des machines virtuelles) pour répondre aux défis du HPC
- Pertinence des Conteneurs dans l'Environnement HPC, article de Xavier et al, qui concluait que LXC est la meilleure technologie de conteneurisation pour l'exécution des calculs HPC.
Ceci leur a permis d'inclure LXC parmi les technologies à comparer
- Travaux de Varma et al. qui les ont permis de comprendre les goulots d'étranglement spécifiques (réseau I/O) rencontrés par Docker
- Travaux de Peinl et al. Au travers desquels ils ont compris que docke présentait sécurité et interopérabilité comme principale faiblesse, et qui par ailleurs étaient la raison directe de l'introduction de Rkt par CoreOS.

Etat de l'art ou contexte et travaux connexes

les auteurs ont réalisé une serie de lectures pour arriver a leurs fins

- la comparaison VM vs conteneurs
- les travaux de Travaux de Chung et al. toujours sur la comparaison VM vs conteneurs, travail qui a légitimé l'accent mis par les auteurs sur l'étude des conteneurs (plutôt que sur une réévaluation des machines virtuelles) pour répondre aux défis du HPC
- Pertinence des Conteneurs dans l'Environnement HPC, article de Xavier et al, qui concluait que LXC est la meilleure technologie de conteneurisation pour l'exécution des calculs HPC.
Ceci leur a permis d'inclure LXC parmi les technologies à comparer
- Travaux de Varma et al. qui les ont permis de comprendre les goulots d'étranglement spécifiques (réseau I/O) rencontrés par Docker
- Travaux de Peinl et al. Au travers desquels ils ont compris que docke présentait sécurité et interopérabilité comme principale faiblesse, et qui par ailleurs étaient la raison directe de l'introduction de Rkt par CoreOS.

Solution proposée

L'article ne propose pas en lui meme une solution, mais plustôt cherchent à déterminer si Rkt, avec sa pile d'exécution différente (notamment l'absence de processus démon, ce qui réduit la surcharge), peut offrir un meilleur support pour les applications HPC intensives en calcul et en données, en fournissant des performances proches du natif trois technologies

- CentOS Rkt
- Docker
- LXC

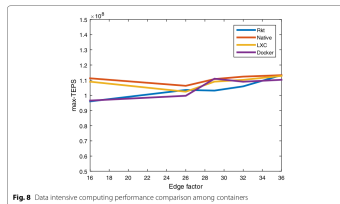
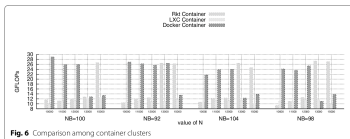
les tests effectués

Deux methodes de benchmarking

- Applications intensives en calcul (CPU-intensive) : Ils utilisent Linpack, un outil qui teste les performances en générant et en résolvant un système d'équations à l'aide de la décomposition LU. La performance est mesurée en Gflops (milliards d'opérations en virgule flottante par seconde). Les tests impliquaient de faire varier la taille du problème (N) et la taille du bloc (NB)
- Applications intensives en données (data-intensive) : Ils utilisent Graph500 pour la génération de données puis effectuent une recherche en largeur sur le graphique construit de celles-ci. La performance est évaluée selon la capacité de suivi des données (Data traceability) et est mesurée en TEPS (Traversed Edges Per Second – Arêtes Traversées Par Seconde)

Les expérimentations ont été menées à la fois sur une configuration à nœud unique et dans un environnement en cluster

les tests effectués



appréciation generale

- Le conteneur CoreOS Rkt offre des performances proches du natif pour les applications intensives tant en données qu'en calcul.
Cette meilleure performance est attribuée non seulement au Le partage du noyau mais aussi à l'absence de processus démon ce qui réduit considérablement la surcharge (overhead) subie par les conteneurs, bien que cela augmente légèrement le temps de démarrage
- Docker donne de meilleures performances pour les petites tailles de problèmes
- Les conteneurs LXC sont plus performants pour les apps intensives en données (proche du natif)

Mon point de vue

Ce que je tire de cet article

- les outils de benchmarking
- le canevas de travail
- les deux approches de comparaison des technologies minimisation des surcharges

An Ultra-lightweight Container that Maximizes memory Sharing and Minimizes the Runtime Environment

Objectif et source de l'article

Objectif de l'article

Cet article de recherche se concentre sur la conception et l'implémentation d'un conteneur ultra-léger visant à maximiser le partage de la mémoire et à minimiser l'environnement d'exécution

Cet article rédigé par Liqing Zhang date de 2019 et a été publié dans le journal International Journal of Science

Contexte et enjeux

L'article s'inscrit dans le contexte de l'essor de la technologie des conteneurs qui a transformé les centres de données. l'article identifie des lacunes importantes de Docker, notamment la redondance du volume des images et le manque d'efficacité dans le partage des ressources. l'enjeu principal était de dépasser les limitations de Docker en matière de redondance de la mémoire et du stockage et de sécurité

Question de recherche

La question de recherche ici est comment concevoir et implémenter une nouvelle architecture de conteneur ultra-léger capable de maximiser le partage des ressources et de minimiser l'environnement d'exécution nécessaire

Etat de l'art

les auteurs ont réalisé une serie de lectures pour arriver a leurs fins

- la comparaison VM vs conteneurs
- les travaux de paul merge et al. : L'existence de ces problèmes (redondance du volume et manque d'efficacité du partage des ressources) a été l'enjeu principal à résoudre
- Ferreira J. B. et Cello M. et al. qui ont confirmé l'importance critique et l'impact du partage des librairies. Ils ont conçu REG pour maximiser le partage des ressources mémoire de l'hôte entre les conteneurs
- Concept Unikernel qui a inspiré les auteurs a abandonner le concept d'image au profit d'une approche où l'application (le code lui-même) est l'image

Etat de l'art

les auteurs ont réalisé une serie de lectures pour arriver a leurs fins

- la comparaison VM vs conteneurs
- les travaux de paul merge et al. : L'existence de ces problèmes (redondance du volume et manque d'efficacité du partage des ressources) a été l'enjeu principal à résoudre
- Ferreira J. B. et Cello M. et al. qui ont confirmé l'importance critique et l'impact du partage des bibliothèques. Ils ont conçu REG pour maximiser le partage des ressources mémoire de l'hôte entre les conteneurs
- Concept Unikernel qui a inspiré les auteurs a abandonner le concept d'image au profit d'une approche où l'application (le code lui-même) est l'image

Etat de l'art

les auteurs ont réalisé une série de lectures pour arriver à leurs fins

- la comparaison VM vs conteneurs
- les travaux de Paul Mergé et al. : L'existence de ces problèmes (redondance du volume et manque d'efficacité du partage des ressources) a été l'enjeu principal à résoudre
- Ferreira J. B. et Cello M. et al. qui ont confirmé l'importance critique et l'impact du partage des bibliothèques. Ils ont conçu REG pour maximiser le partage des ressources mémoire de l'hôte entre les conteneurs
- Concept Unikernel qui a inspiré les auteurs à abandonner le concept d'image au profit d'une approche où l'application (le code lui-même) est l'image

Etat de l'art

les auteurs ont réalisé une série de lectures pour arriver à leurs fins

- la comparaison VM vs conteneurs
- les travaux de Paul Mergé et al. : L'existence de ces problèmes (redondance du volume et manque d'efficacité du partage des ressources) a été l'enjeu principal à résoudre
- Ferreira J. B. et Cello M. et al. qui ont confirmé l'importance critique et l'impact du partage des bibliothèques. Ils ont conçu REG pour maximiser le partage des ressources mémoire de l'hôte entre les conteneurs
- Concept Unikernel qui a inspiré les auteurs à abandonner le concept d'image au profit d'une approche où l'application (le code lui-même) est l'image

Les tâches à faire

Les tâches a faire

Pour la semaine prochaine

- Termine La lecture et le résumé de l'article " An Ultra-lightweight Container that Maximizes memory Sharing and Minimizes the Runtime Environment"
- La lecture et le résumé de l'article sur singularity
- proposer une architecture du runtime a créer.
- continuer l'apprentissage du langage rust.

Les tâches a faire

Pour la semaine prochaine

- Termine La lecture et le résumé de l'article " An Ultra-lightweight Container that Maximizes memory Sharing and Minimizes the Runtime Environment"
- La lecture et le résumé de l'article sur singularity
- proposer une architecture du runtime a créer.
- continuer l'apprentissage du langage rust.

Les tâches a faire

Pour la semaine prochaine

- Termine La lecture et le résumé de l'article " An Ultra-lightweight Container that Maximizes memory Sharing and Minimizes the Runtime Environment"
- La lecture et le résumé de l'article sur singularity
- proposer une architecture du runtime a créer.
- continuer l'apprentissage du langage rust.

Les tâches a faire

Pour la semaine prochaine

- Termine La lecture et le résumé de l'article " An Ultra-lightweight Container that Maximizes memory Sharing and Minimizes the Runtime Environment"
- La lecture et le résumé de l'article sur singularity
- proposer une architecture du runtime a créer.
- continuer l'apprentissage du langage rust.

Merci !!!